



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Ingegneria
“Enzo Ferrari”

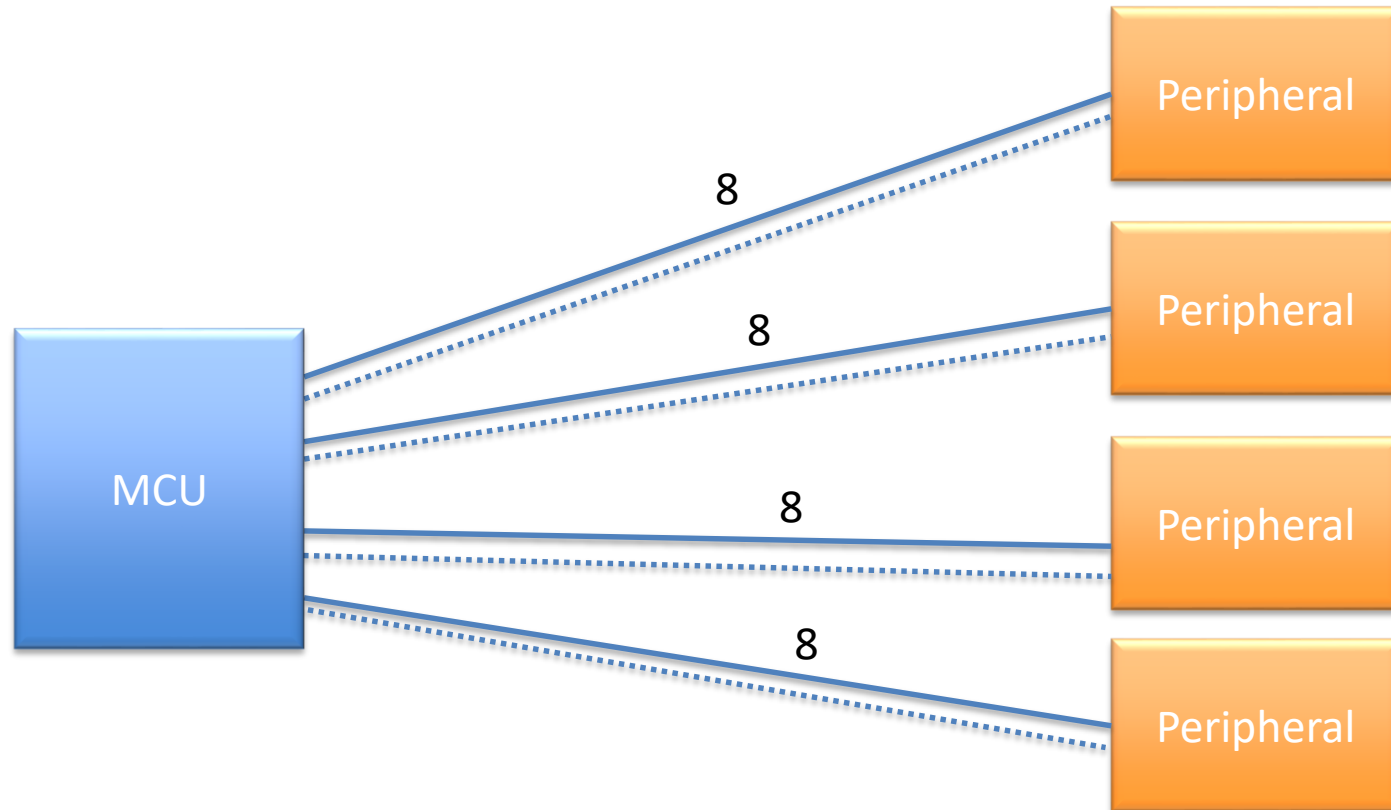
Fondamenti di Elettronica Digitale

Wired Protocols

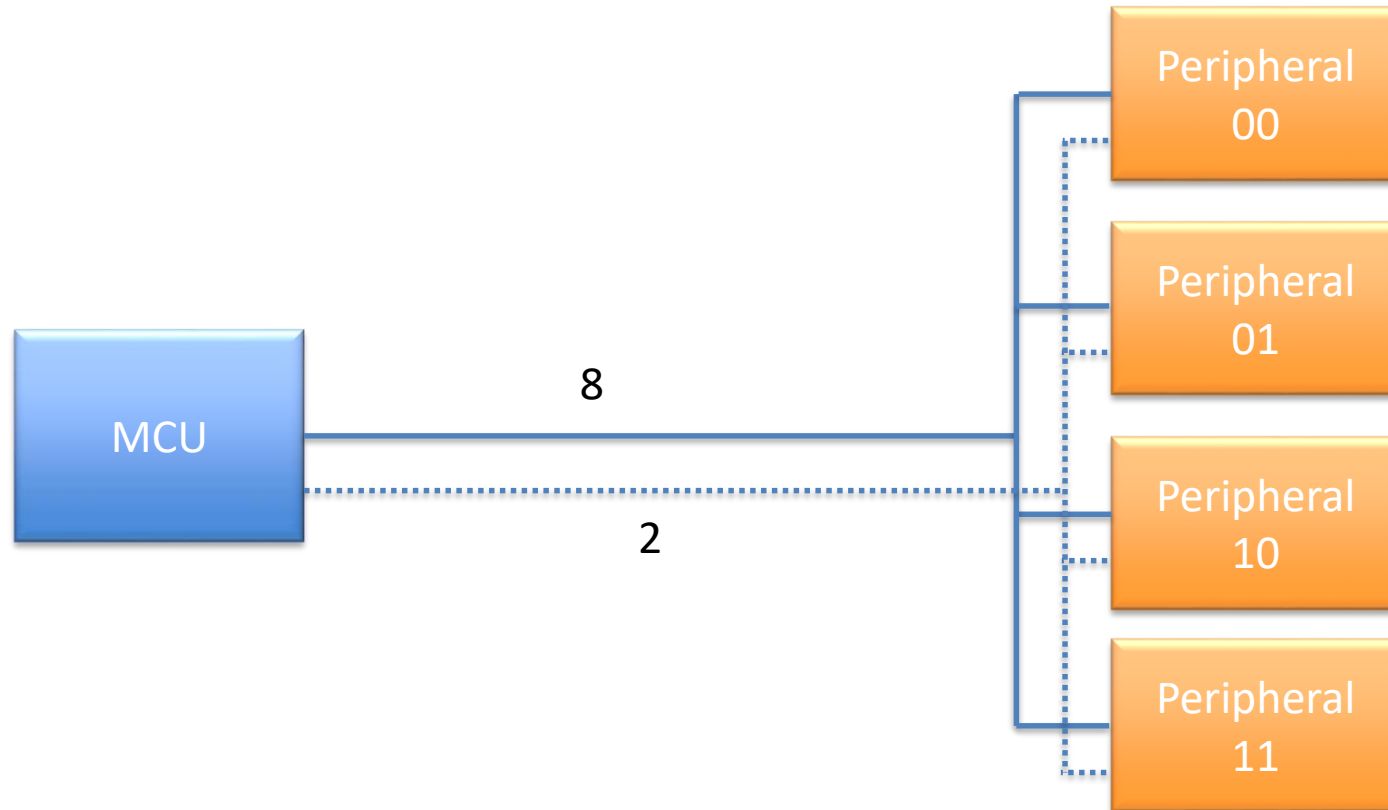
Laurea Triennale in Ingegneria Informatica

Prof. Francesco Maria Puglisi
email: francescomaria.puglisi@unimore.it
phone: +39-059-2056352

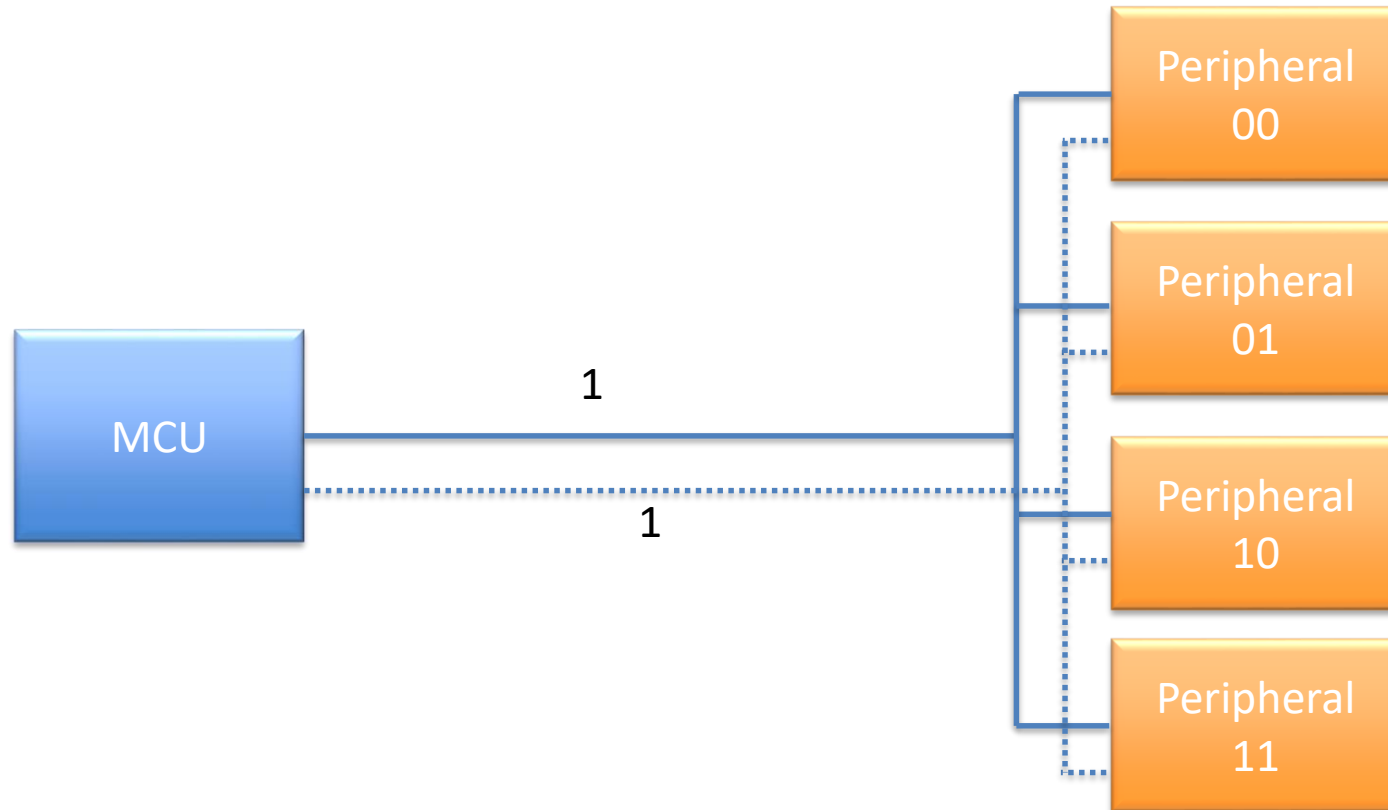
Need of Serial Bus Protocol



Need of Serial Bus Protocol



Need of Serial Bus Protocol



Need of Serial Bus Protocol

- Peripheral devices in embedded systems => parallel address and data bus => lots of wiring and requires number of pins => additional decoding logic required.
- To reduce the pins and wiring => cost => Serial bus protocol => SPI (4-wire) & I2C (2-wire).
- Penalty => Slower communication.

Various Serial Bus Protocol

- UART
- SPI –Embedded System Protocol
- I2C- Embedded System Protocol
- CAN
- USB
- SATA
- etc...

Serial Peripheral Interface (SPI)

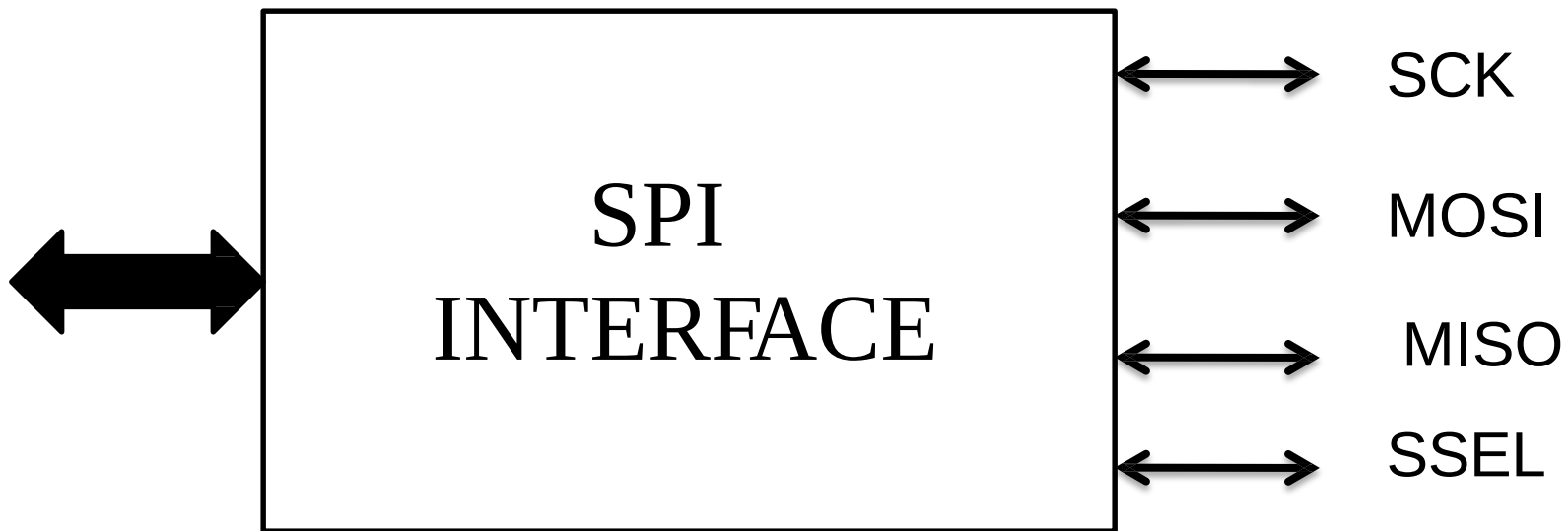
- The Serial Peripheral Interface Bus or SPI bus is a synchronous serial data link standard named by Motorola that operates in full duplex mode.
- Devices communicate in master/slave mode where the master device initiates the data frame. Multiple slave devices are allowed with individual slave select (chip select) lines.

- During a data transfer the master always sends 8 to 16 bits of data to the slave, and the slave always sends a byte of data to the master.
- Maximum data bit rate is one eighth of the input clock rate.

- One Central device (Master), initiates communication with all slaves.
- No address decoding logic required.
- SPI Master wishes to send the data to slave or request information from the slave, it activates the clock signal.
- Master generates information on one line (MOSI) while samples (read) from another line (MISO).

SPI Pin Description

- SCLK — Serial Clock (output from master)
- MOSI — Master Output, Slave Input (output from master)
- MISO — Master Input, Slave Output (output from slave)
- SS — Slave Select (active low; output from master)



Pin Name : SCK (Serial Clock)

Type : Input / Output

- The SPI used clock signal to synchronize the transfer of data across the SPI interface.
- The SCK is always driven by the master and received by the slave, The clock is programmable to be active high or active low.
- The SCK is only active during a data transfer. Any other time, it is either in its inactive state, or tri-stated.

Pin Name : MISO

(Master in Slave out)

Type : Input / Output

- The MISO signal is a unidirectional signal used to transfer serial data from the slave to the master.
- When a device is a slave, serial data is output on this signal.
- When a device is a master, serial data is input on this signal.
- When a slave device is not selected, the slave drives the signal high impedance.

Pin Name : **MOSI**
(Master out Slave in)

Type : **Input / Output**

- The MOSI signal is a unidirectional signal used to transfer serial data from the Master to the Slave.
- When a device is a Master, serial data is output on this signal.
- When a device is a Slave, serial data is input on this signal.

Pin Name : SSEL (Slave Select)

Type : Input

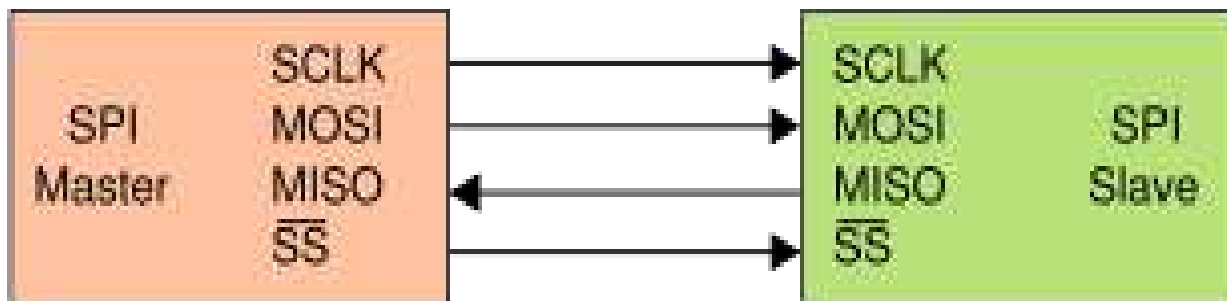
- The SPI slave select signal is an active low signal that indicates which slave is currently selected to participate in a data transfer.
- Each slave has its own unique slave select signal input.
- The SSEL must be low before data transactions begin and normally stays low for the duration of the transaction.
- If the SSEL signal goes high any time during a data transfer, the transfer is considered to be aborted.

- In this event, the slave returns to idle, and any data that was received is thrown away. There are no other indications of this exception.

- This signal is not directly driven by the master. It could be driven by a simple general purpose I/O under software control.

Operation

- The SPI bus can operate with a single master device and with one or more slave devices.
- SPI bus: single master and single slave



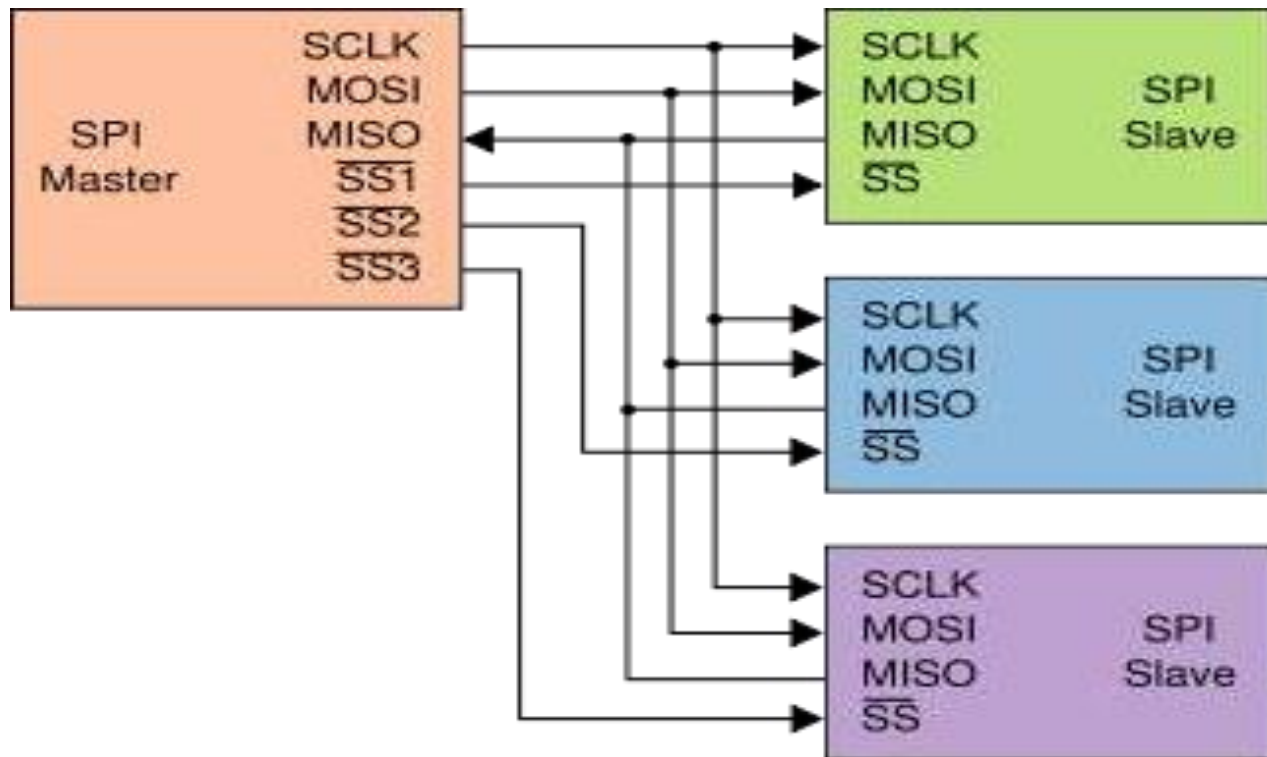
- If a single slave device is used, the [SSEL](#) pin *may* be fixed to [logic low](#) if the slave permits it.
- Some slaves require the [falling edge](#) (high->low transition) of the [slave select](#) to initiate an action such as the MAX1242 by [Maxim](#), an [ADC](#), that starts conversion on said transition.

Configuration

- Two types multiple slave configuration:
- Typical SPI bus: Master and independent Slaves
- Daisy-Chained SPI bus: Master and cooperative slaves

Typical SPI Bus

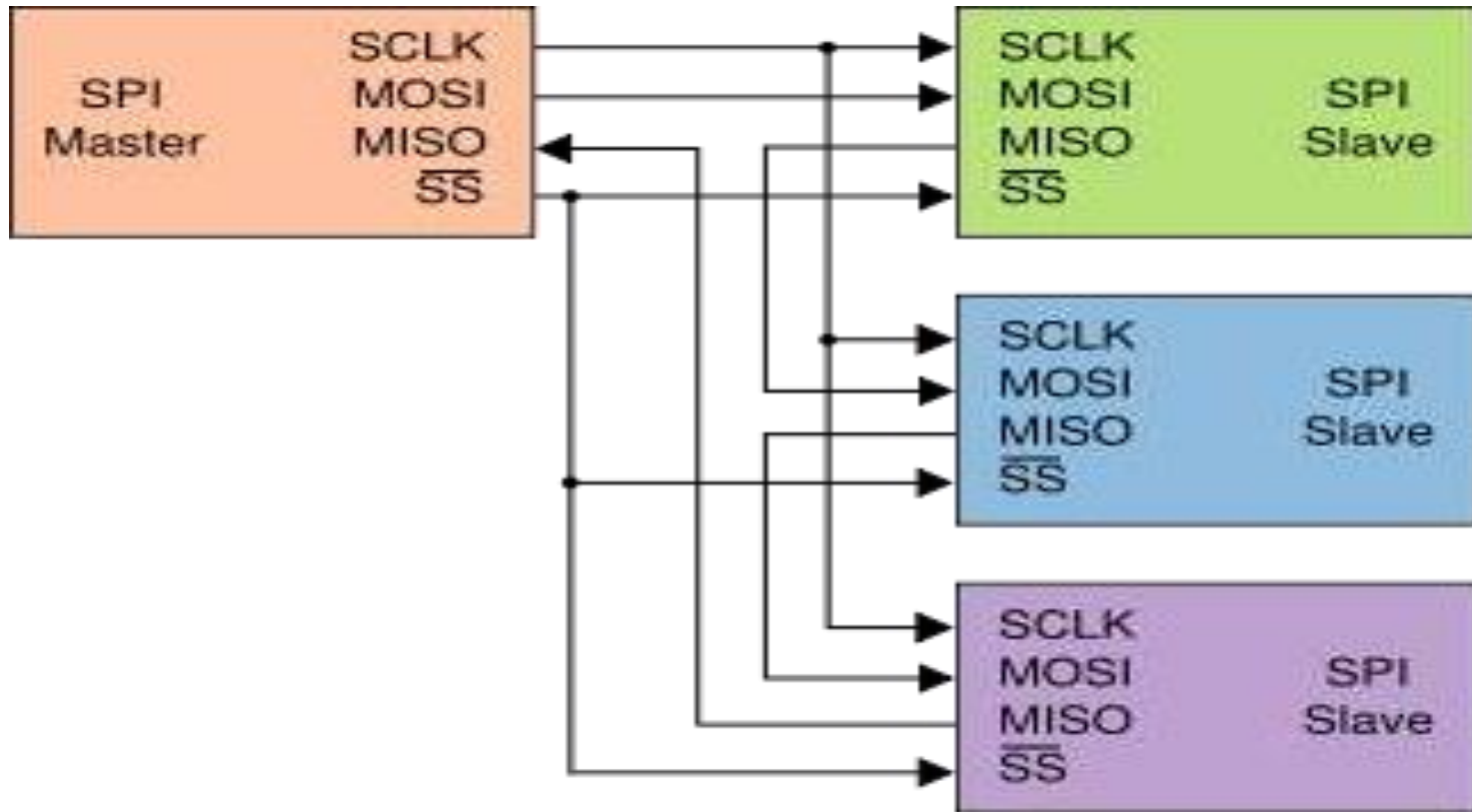
- With multiple slave devices, an independent SSEL signal is required from the master for each slave device (3).



- In the independent slave configuration, there is an independent slave select line for each slave. This is the way SPI is normally used.

- Since the MISO pins of the slaves are connected together, they are required to be tri-state pins.

Daisy-Chained SPI Bus



Daisy-chained SPI bus: Master and Cooperative Slaves

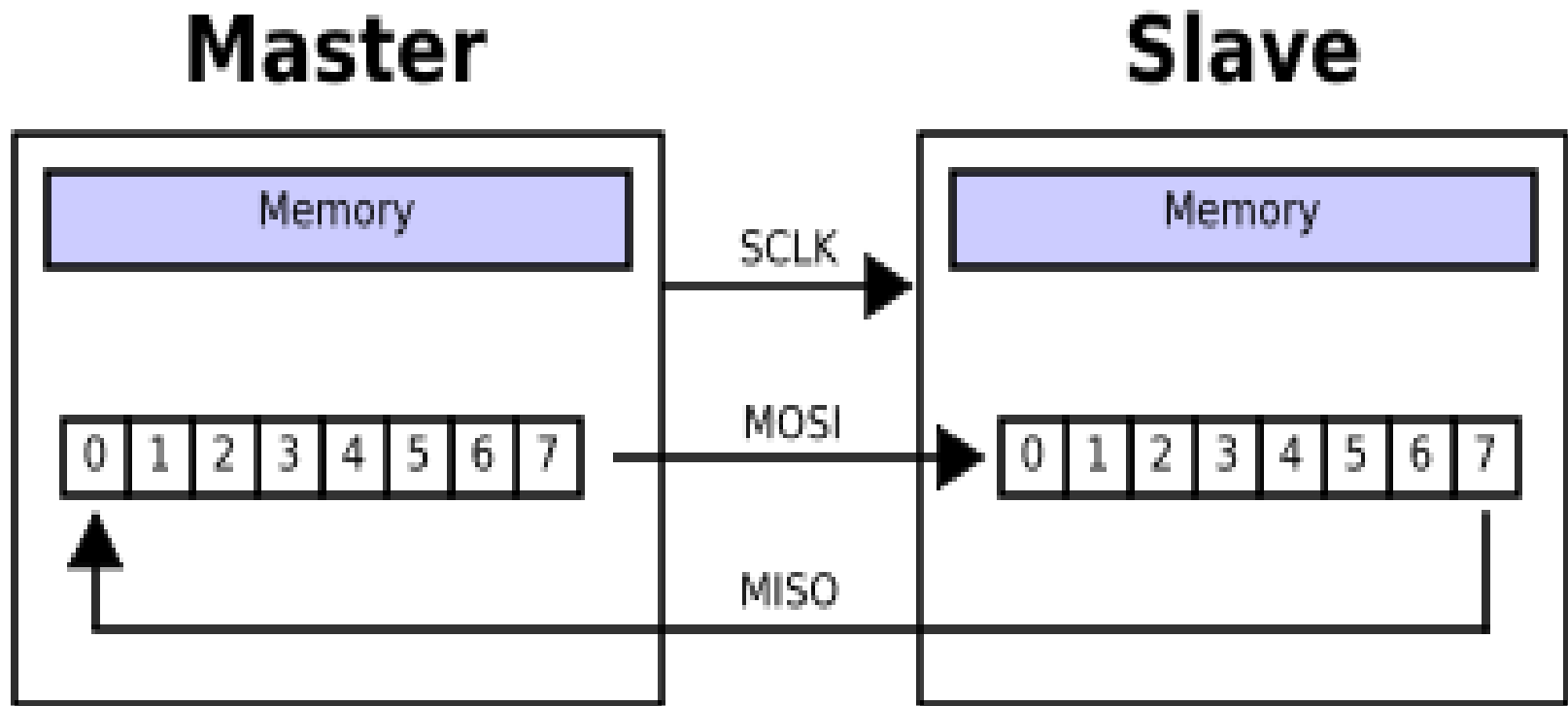
- Some products with SPI bus are designed to be capable of being connected in a daisy chain configuration, the first slave output being connected to the second slave input, etc.
- The SPI port of each slave is designed to send out during the second group of clock pulses an exact copy of what it received during the first group of clock pulses.
- Such a feature only requires a single SSEL line from the master, rather than a separate SSEL line for each slave.

Points

- Not have ack mechanism to confirm receipt of data and does not have flow control.
- SPI Master, not have knowledge of whether slave exist or Not
- Not particular addressing scheme.
- Not defined any maximum data rate.

Data Transmission

- A typical hardware setup using two shift registers to form an inter-chip circular buffer



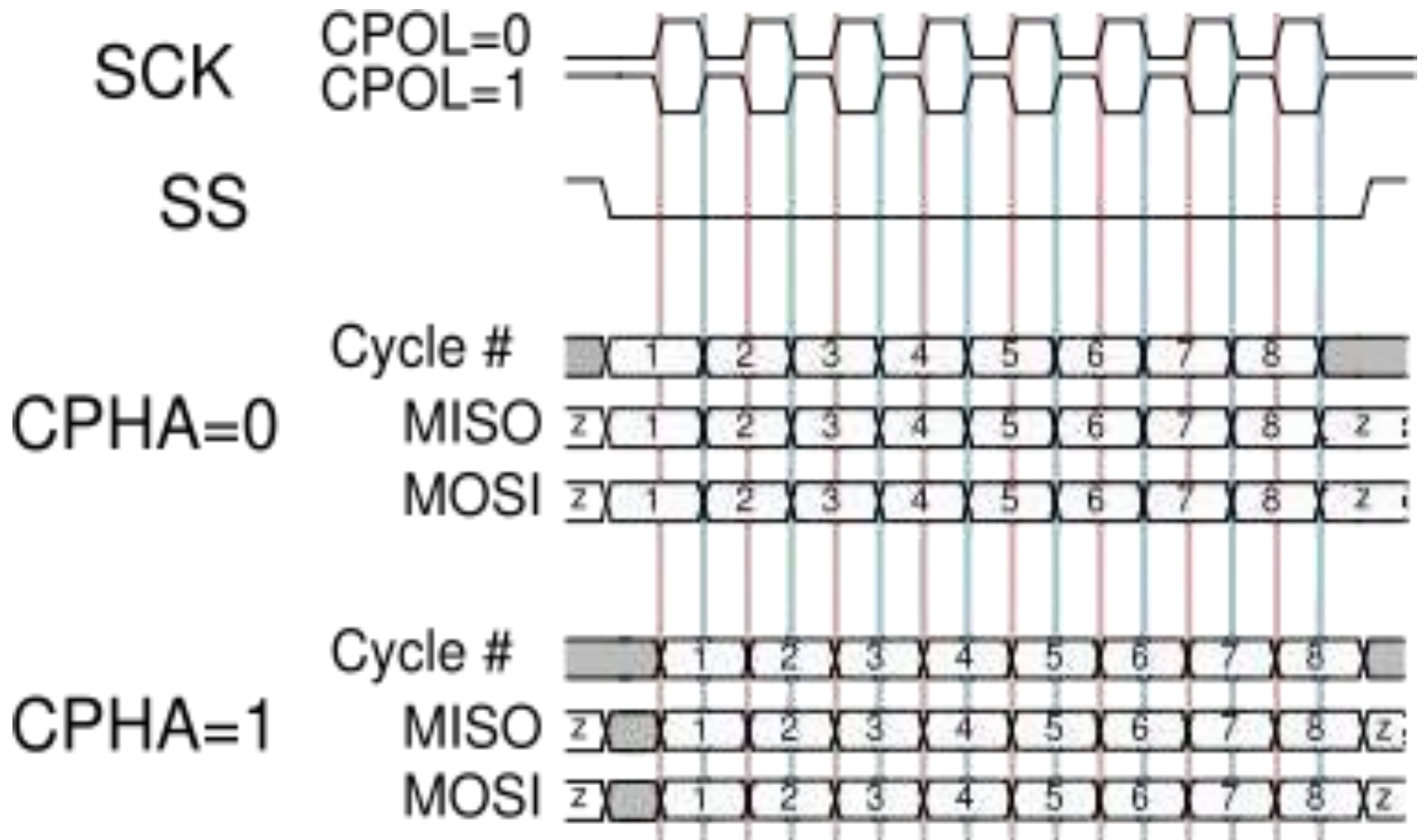
- To begin a communication, the master first configures the [Clock](#), using a frequency less than or equal to the maximum frequency the slave device supports.
- Such frequencies are commonly in the range of [1-70 MHz](#).
- The master then pulls the slave select [SSEL low](#) for the desired chip.
- During each SPI clock cycle, a [full duplex](#) data transmission occurs.

- The master sends a bit on the MOSI line; the slave reads it from that same line
- The slave sends a bit on the MISO line; the master reads it from that same line
- Transmissions normally involve two shift registers of some given word size, such as eight bits, one in the master and one in the slave; they are connected in a ring.

- After that register has been shifted out, the master and slave have exchanged values.
- Then each device takes that value and does something with it, such as writing it to memory.
- If there are more data to exchange, the shift registers are loaded with new data and the process repeats.

Clock Polarity and Phase

- In addition to setting the clock frequency, the **master** must also configure the clock polarity and phase with respect to the **data**.
- SPI Block Guide names these two options as **CPOL** **and CPHA** respectively, and most vendors have adopted that convention.



A timing diagram showing clock polarity and phase

At CPOL=0, the base value of the clock is zero

- For CPHA=0, data are read on the clock's rising edge (low->high transition) and data are changed on a falling edge (high->low clock transition).
- For CPHA=1, data are read on the clock's falling edge and data are changed on a rising edge.

At CPOL=1, the base value of the clock is one
(inversion of CPOL=0)

- For CPHA=0, data are read on clock's falling edge and data are changed on a rising edge.
- For CPHA=1, data are read on clock's rising edge and data are changed on a falling edge.

CPOL & CPHA	First data driven	Other data driven	Data Sampled
0 & 0	Prior to first SCK rising edge	SCK falling edge	SCK rising edge
0 & 1	First SCK rising edge	SCK rising edge	SCK falling edge
1 & 0	Prior to first SCK falling edge	SCK rising edge	SCK falling edge
1 & 1	First SCK falling edge	SCK falling edge	SCK rising edge

Mode Numbers

- The combinations of polarity and phases are often referred to as modes

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Register Description

- SPI has seven registers, from that programmers interface for SPI peripheral has five registers.
- The bits in the rest of two TEST registers are intended for functional verification only.

Name	Description	Access
S0SPCR	<u>SPI Control Register.</u> This register controls the operation of the SPI.	R/W
S0SPSR	<u>SPI Status Register.</u> This register shows the status of the SPI.	R0

Name	Description	Access
S0SPDR	<u>SPI Data Register.</u> This bi-directional register provides the transmit and receive data for the SPI.	R/W

Name	Description	Access
S0SPCCR	<u>SPI Clock Counter Register.</u> This register controls the frequency of a master's SCK	R/W
S0SPINT	<u>SPI Interrupt Flag.</u> This register contains the interrupt flag for the SPI interface.	R/W

(1) SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
-----	--------	-------	-------------

<u>1:0</u>	-		Reserved, user software should not write ones to reserved bits.
------------	---	--	---

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
-----	--------	-------	-------------

<u>2</u>	BitEnable	0	The SPI controller sends and receives 8 bits of data per transfer.
----------	-----------	---	--

<u>1</u>			The SPI controller sends and receives the number of bits selected by bits 11:8 .
----------	--	--	--

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
<u>3</u>	CPHA		Clock phase control
		0	Data is sampled on the first clock edge of SCK.
		1	Data is sampled on the second clock edge of the SCK.

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
<u>4</u>	CPOL		Clock polarity control.
		0	SCK is active high.
		1	SCK is active low.
<u>5</u>	MSTR		Master mode select.
		<u>0</u>	The SPI operates in Slave mode.
		<u>1</u>	The SPI operates in Master mode.

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
<u>6</u>	LSBF		<u>LSB First</u> , controls in which <u>direction</u> each byte is shifted when transferred.
		0	SPI data is transferred MSB (bit 7) first.
		<u>1</u>	SPI data is transferred LSB (bit 0) first.

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
<u>7</u>	SPIE		Serial peripheral interrupt enable.
		0	SPI interrupts are inhibited.
		<u>1</u>	A hardware interrupt is generated each time the <u>SPIF</u> or MODF bits are activated.

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
<u>11:8</u>	BITS		When bit 2 of this register is 1, this field controls the number of bits per transfer:
		<u>1000</u>	<u>8 bits per transfer</u>
		1001	9 bits per transfer
		1010	10 bits per transfer
		1011	11 bits per transfer

SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
-----	--------	-------	-------------

		1100	12 bits per transfer
--	--	------	----------------------

		1101	13 bits per transfer
--	--	------	----------------------

		1110	14 bits per transfer
--	--	------	----------------------

		1111	15 bits per transfer
--	--	------	----------------------

		<u>0000</u>	<u>16 bits per transfer</u>
--	--	-------------	-----------------------------

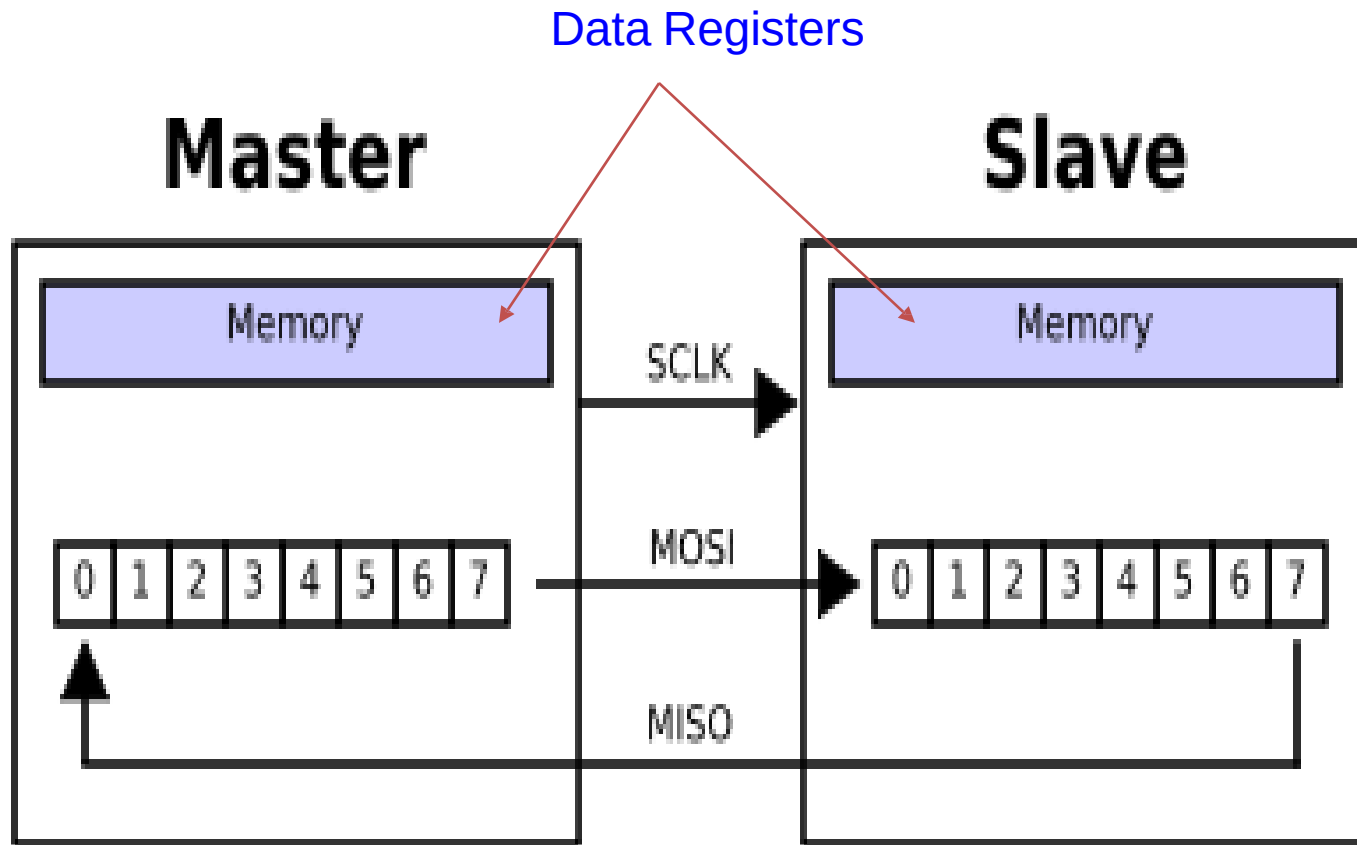
SPI Control Register (S0SPCR - 0xE002 0000)

Bit	Symbol	Value	Description
-----	--------	-------	-------------

<u>15:12</u>	-		Reserved, user software should not write ones to reserved bits.
--------------	---	--	---

(2) SPI Data Register (S0SPDR - 0xE002 0008)

- This bi-directional data register provides the transmit and receive data for the SPI.
- Transmit data is provided to the SPI by writing to this register.
- Data received by the SPI can be read from this register.



- There is no buffer between the data register and the internal shift register. A write to the data register goes directly into the internal shift register.
- Therefore, data should only be written to this register when a transmit is not currently in progress.

- Read data is buffered.
- When a transfer is complete, the receive data is transferred to a single byte data buffer, where it is later read.
- A read of the SPI data register returns the value of the read data buffer.

SPI Data Register (S0SPDR - 0xE002 0008)

Bit	Symbol	Description
<u>7:0</u>	<u>DataLow</u>	SPI <u>bi-directional</u> data port.
<u>15:8</u>	<u>DataHigh</u>	If <u>bit 2</u> of the SPCR is 1 and bits 11:8 are other than 1000, some or all of these bits contain the additional transmit and receive bits. When <u>less than 16 bits</u> are <u>selected</u> , the most significant among these bits <u>read as zeroes</u> .

(3) SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
-----	--------	-------------

<u>7</u>	SPIF	<u>SPI transfer complete flag.</u>
----------	------	------------------------------------

When 1, this bit indicates when a SPI data transfer is complete.

When a master, this bit is set at the end of the last cycle of the transfer.

SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
-----	--------	-------------

<u>7</u>	SPIF	<u>SPI transfer complete flag.</u>
----------	------	------------------------------------

When a slave, this bit is set on the last data sampling edge of the SCK.

This bit is cleared by first reading this register then accessing the SPI data register.

SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
-----	--------	-------------

<u>6</u>	WCOL	Write Collision . When 1 , this bit indicates that a write collision has occurred. This bit is cleared by reading this register then accessing the SPI data register.
----------	------	---

Exception conditions –Write Collision

- As stated previously, there is no write buffer between the SPI block bus interface, and the internal shift register.
- As a result, data must not be written to the SPI data register when a SPI data transfer is currently in progress.

- The time frame where data cannot be written to the SPI data register is from when the transfer starts, until after the status register has been read when the SPIF status is active.

- If the SPI data register is written in this time frame, the write data will be lost, and the write collision (WCOL) bit in the status register will be activated.

SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
-----	--------	-------------

<u>5</u>	ROVR	Read overrun. When 1 , this bit indicates that a read overrun has occurred. This bit is cleared by reading this register.
----------	------	---

Exception conditions –Read Overrun

- A read overrun occurs when the SPI block internal read buffer contains data that has not been read by the processor, and a new transfer is completed.
- The read buffer containing valid data is indicated by the SPIF bit in the status register being active.

Exception conditions –Read Overrun

- When a transfer completes, the SPI block needs to move the received data to the read buffer.
- If the SPIF bit is active (the read buffer is full), the new receive data will be lost, and the read overrun (ROVR) bit in the status register will be activated.

SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
-----	--------	-------------

<u>4</u>	MODF	<u>Mode fault</u> . when 1, this bit indicates that a Mode fault error has occurred. This bit is cleared by reading this register, then writing the SPI Control register.
----------	------	---

Exception conditions –Mode Fault

- If the SSEL signal goes active, when the SPI block is a master, this indicates another master has selected the same device to be a slave. This condition is known as a mode fault.
- When a mode fault is detected, the mode fault (MODF) bit in the status register will be activated.

SPI Status Register (S0SPSR - 0xE002 0004)

Bit	Symbol	Description
<u>3</u>	ABRT	<u>Slave abort</u> . When 1, this bit indicates that a slave abort has occurred. This bit is cleared by reading this register.
<u>2:0</u>	-	Reserved, user software should not write ones to reserved bits.

Exception conditions –Slave Abort

- A slave transfer is considered to be aborted, if the SSEL signal goes inactive before the transfer is complete.
- In the event of a slave abort, the transmit and receive data for the transfer that was in progress are lost, and the slave abort(ABRT) bit in the status register will be activated.

SPI Interrupt Register (S0SPINT - 0xE002 001C)

- This register contains the interrupt flag for the SPI interface.

Bit	Symbol	Description
<u>0</u>	SPI Interrupt Flag	SPI interrupt flag. <u>Set by the SPI interface to generate an interrupt. Cleared by writing a 1 to this bit.</u>
<u>7:1</u>	-	Reserved, user software should not write ones to reserved bits.

SPI Clock Counter Register (S0SPCCR - 0xE002 000C)

- This register controls the frequency of a master's SCK.
- The register indicates the number of PCLK cycles that make up an SPI clock.
- The value of this register must always be an even number. As a result, bit 0 must always be 0.

Configuration

- SPI can be configured as MASTER or SLAVE.

Configuration - Master operation

- The following sequence describes how one should process a data transfer with the SPI block when it is set up to be the master.
- This process assumes that any prior data transfer has already completed.

Configuration - Master operation

1. Set the [SPI Clock counter](#) register to the desired clock rate.
2. Set the [SPI Control register](#) to the desired settings.
3. Write the data to be transmitted to the [SPI data register](#). This write [starts](#) the SPI data transfer.

Configuration - Master operation

4. Wait for the SPIF bit in the SPI status register to be set to 1. The SPIF bit will be set after the last cycle of the SPI data transfer.
5. Read the SPI status register.

Configuration - Master operation

6. Read the [received data](#) from the [SPI data register](#) (optional).

7. Go to [step 3](#) if more data is required to transmit.

Configuration - Master operation

NOTE:

- A read or write of the SPI data register is required in order to clear the SPIF status bit.
- Therefore, if the optional read of the SPI data register does not take place, a write to this register is required in order to clear the SPIF status bit.

Configuration - Slave operation

- The following sequence describes how one should process a data transfer with the SPI block when it is set up to be the slave.
- This process assumes that any prior data transfer has already completed.

Configuration - Slave operation

1. Set the [SPI control register](#) to the desired settings.
2. Write the data to be transmitted to the [SPI data register](#) (optional). Note that this can only be done when a slave SPI transfer is [not](#) in progress.

Configuration - Slave operation

3. Wait for the SPIF bit in the SPI status register to be set to 1. The SPIF bit will be set after the last sampling clock edge of the SPI data transfer.
4. Read the SPI status register.

Configuration - Slave operation

5. [Read](#) the received data from the [SPI data register](#) (optional).
6. Go to [step 2](#) if more data is required to transmit.

Configuration - Slave operation

NOTE:

- A read or write of the SPI data register is required in order to clear the SPIF status bit.
- Therefore, at least one of the optional reads or writes of the SPI data register must take place, in order to clear the SPIF status bit.

Advantages

- Full duplex communication
- Higher throughput than I²C
- Complete protocol flexibility for the bits transferred
 - * Not limited to 8-bit words
 - * Arbitrary choice of message size, content, and purpose

Advantages

- Extremely simple hardware interfacing
 - * Typically lower power requirements than I²C due to less circuitry
 - * No arbitration or associated failure modes
 - * Slaves use the master's clock, and don't need precision oscillators
 - * Transceivers are not needed

Disadvantages

- Requires more pins than I²C
- No hardware [flow control](#)
- No hardware slave acknowledgment (the master could be "talking" to nothing and not know it)

Disadvantages

- Supports only one master device
- Only handles short distances compared to [RS-232](#), [RS-485](#), or [CAN-bus](#)

Applications

SPI is used to talk to a variety of peripherals, such as:

- Sensors: Temperature, pressure, [ADC](#), touch-screens
- Control devices: [audio codecs](#), digital potentiometers, [DAC](#)

Applications

- Memory: [flash](#) and EEPROM
- Real-time clocks
- LCD displays, sometimes even for managing image data
- Any [MMC](#) or [SD](#) card

Inter-Integrated Circuit (I²C)

I²C/ TWI

I2C or I²C came from the term Inter Integrated Circuit.

Since I2C is implemented by only Two Wires (SDA, SCL), it is also called TWI (Two Wire Interface).

I2C is a low-bandwidth, short distance protocol for on board communications.

All devices are connected through two wires: serial data (SDA) and serial clock (SCL).

I²C/ TWI

- When connecting multiple devices to a microcontroller, the address and data lines of each device were conventionally connected individually.
- This would take up precious pins on the microcontroller, result in a lot of traces on the PCB, and require more components to connect everything together.
- This made these systems expensive to produce and susceptible to interference and noise.

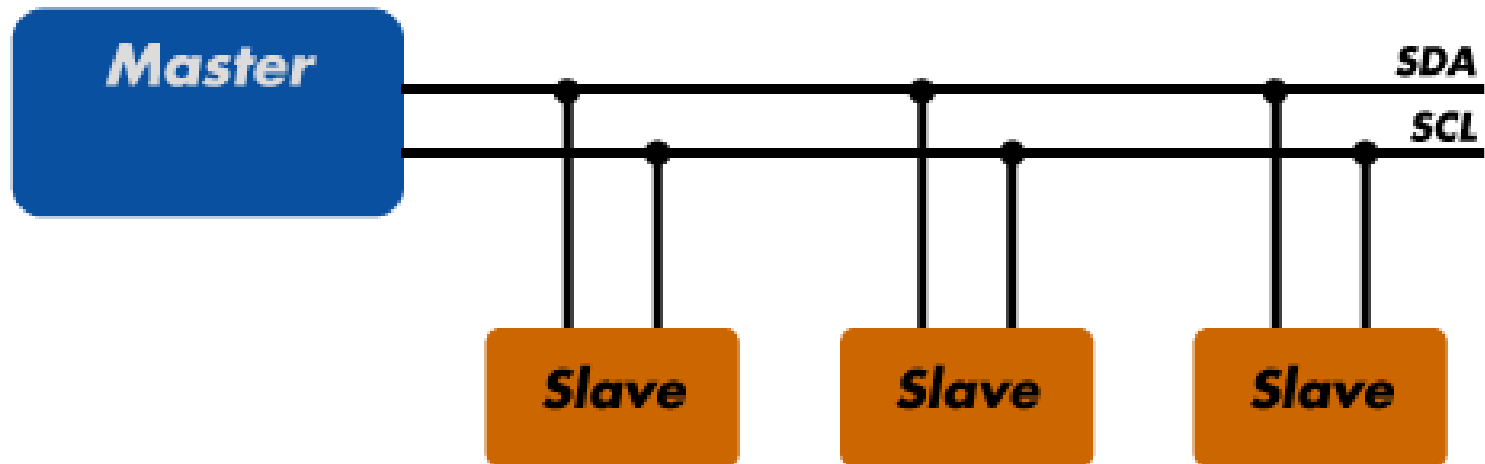
Connections in I²C network

- It is a kind of serial communication technology which is originally designed for exchanging data between multiple IC chips.
- This is very simple communication mechanism and very smartly designed.
- It requires only two lines.
- One of the line is used for sending Clock signal and the other line is to send/receive data.
- Also, these two lines can be shared by multiple devices. This communication works as Master-Slave mode. One Master can control / communicate multiple Slaves.

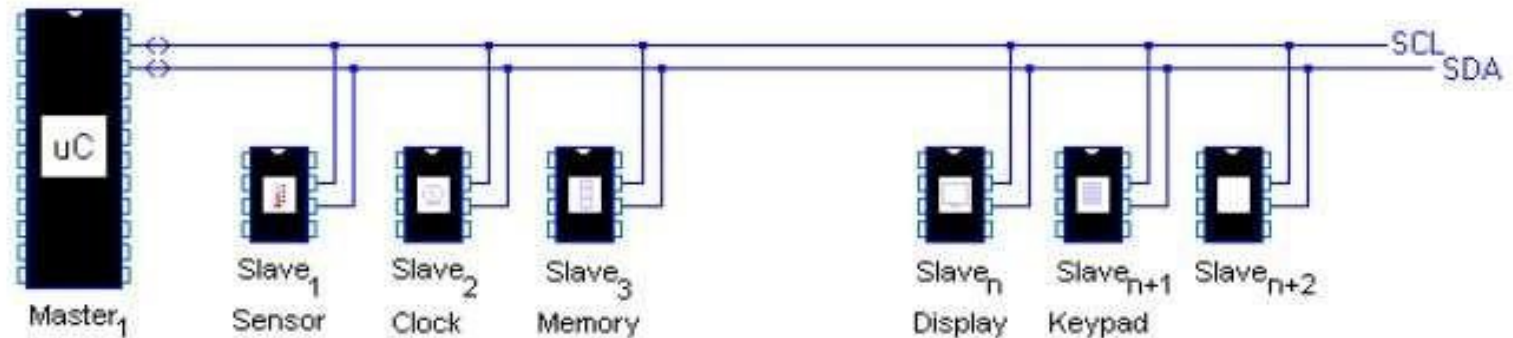
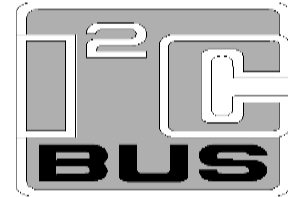
Connections in I²C network

- Regardless of how many slave units are attached to the I2C bus, there are only two signals connected to all of them.
- Consequently, there is additional overhead because an addressing mechanism is required for the master device to communicate with a specific slave device.
- Because all communication takes place on only two wires, all devices must have a unique address to identify it on the bus.
- Slave devices have a predefined address, but the lower bits of the address can be assigned to allow for multiples of the same devices on the bus.

Multiple Devices



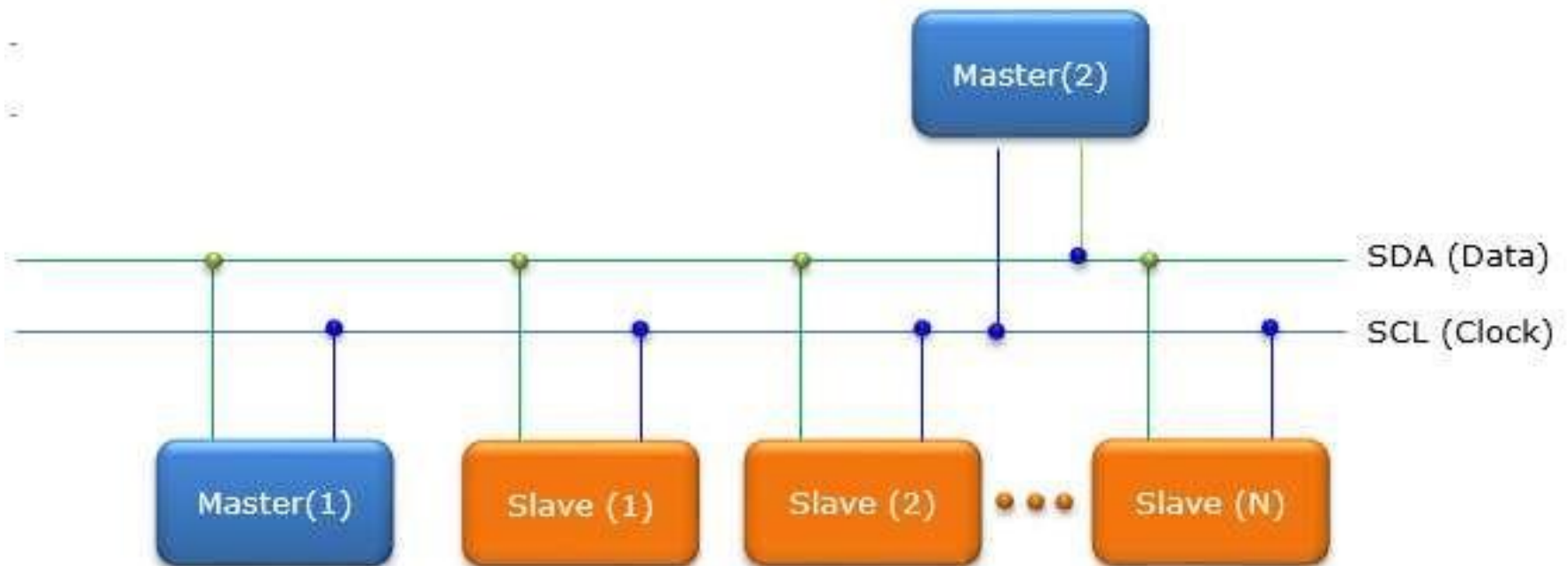
Multiple Devices



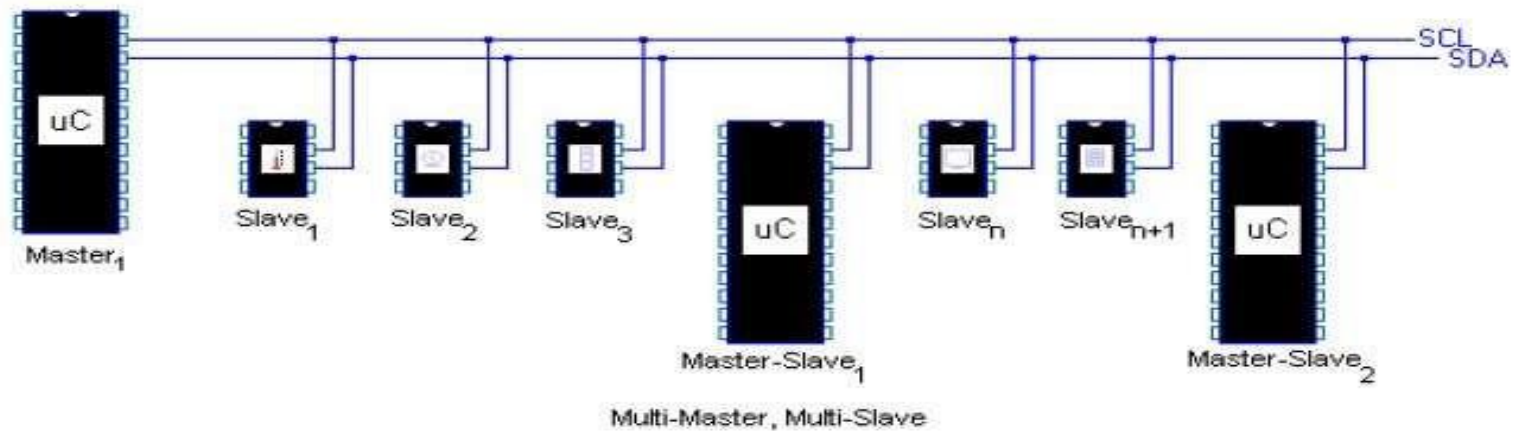
One I²C Master, Multiple Slaves

- Master :- Initiate a message
- Slave :- Response to message

Multi-Master Support



Multi-Master Support

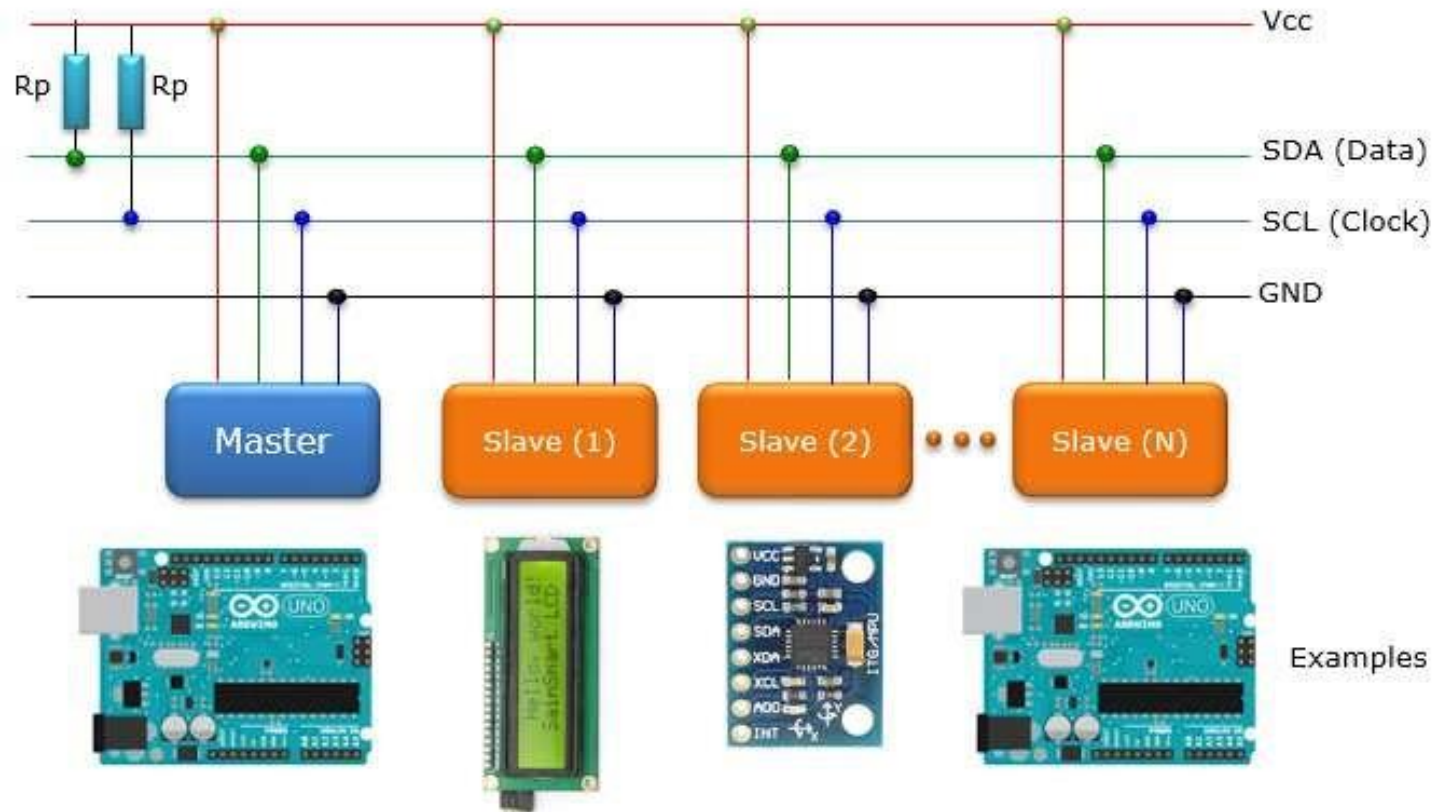


- Master detect arbitration loss

Multi-Master Support

- Even though you need only two lines in I2C, in real application you would need a couple of more lines because we need to supply Power to each of the connected chipset and common ground as shown below.
- Because of this, you would see 4 wires attached to a device (small I2C modules) that use I2C.

Examples



Theory of Operation

- I2C has a master/slave protocol. The master initiates the communication. The sequence of events are:
 - ✓ The Master device issues a start condition. This condition informs all the slave devices to listen on the serial data line for instructions.
 - ✓ The Master device sends the address of the target slave device and a read/write flag.
 - ✓ The Slave device with the matching address responds with an acknowledgement signal.
 - ✓ Communication proceeds between the Master and the Slave on the data bus. Both the master and slave can receive or transmit data depending on whether the communication is a read or write. The transmitter sends 8-bits of data to the receiver which replies with a 1-bit acknowledgement.
 - ✓ When the communication is complete, the master issues a stop condition indicating that everything is done.

I²C Communication Protocol

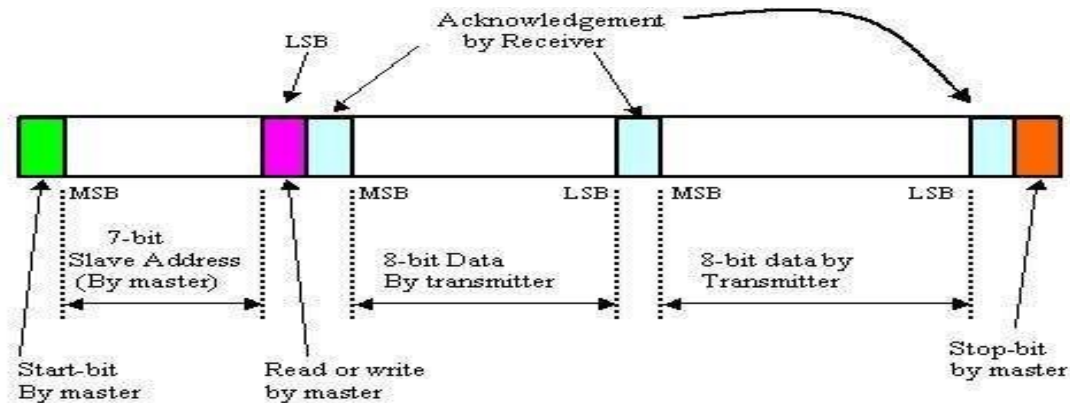


Fig-2 (I²C communication protocol)



I²C Communication Protocol

Start & stop condition

- Start Condition – A high to low transition on SDA when SCL is high.
- Stop Condition – A low to high transition on SDA when SCL is high.

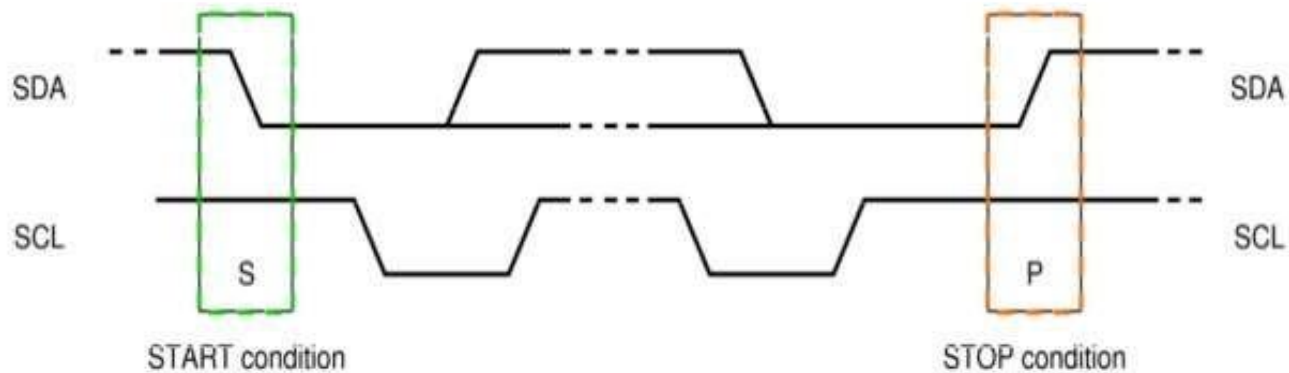


Fig 3

Features

- ✓ It supports multiple data speeds: standard (100 kbps), fast (400 kbps) and high speed (3.4 Mbps) communications.
- ✓ Built in collision detection
- ✓ 10-bit Addressing
- ✓ Multi-master support
- ✓ Data broadcast (general call)

Advantages

- Only two signal lines requires
- Flexible data transmission rates
- Each device on the bus is independently addressable
- Devices have a simple Master/Slave relationship
- Capable of handling multiple master communications by providing arbitration and communication collision detection

Disadvantages

- Open Collector driver at master needs pull up resistance 2.2k on each line
- High Power Requirement
- Low Speed
- Low Throughput

Comparison : I²C vs. SPI

I ² C	SPI
Requires only two lines	Requires a minimum of four lines
Low Speed	Higher Speed
Half Duplex	Full Duplex
Additional SS lines are NOT required as the number of devices increases	Additional SS lines are required as the number of devices increases
More Power Required	Less Power Required
Multi-master can be implemented easily	Multi-master is difficult to implement